

Course ID : CSE 210

Group : 03

Assignment : 4-bit MIPS Design and Simulation

Course Name : Computer Architecture Sessional

Date of Submission : 19 September, 2026

Department : CSE

Level-Term : L2-T2

Section : B1

Group : 3

Group Member's ID :

2305065

2305066

2305069

2305079

2305085

Problem Specification

We need to design and simulate a reduced 16-bit-instruction MIPS processor with:

- A 4-bit data path
- An 8-bit address bus
- Temporary 4-bit registers

MIPS instruction format:

R-type	Opcode	Src Reg 1	Src Reg 2	Dst Reg
	4 bits	4 bits	4 bits	4 bits

S-type	Opcode	Src Reg 1	Dst Reg	Shamt
	4 bits	4 bits	4 bits	4 bits

I-type	Opcode	Src Reg 1	Src 2 / Dst Reg	Addr. / Immdt.
	4 bits	4 bits	4 bits	4 bits

J-type	Opcode	Target Jump Address	0
	4 bits	8 bits	4 bits

For B1, Group 3, the Opcode sequence is:

O J N C A E P L G K M F D B H I

So, our instruction set looks like the one below:

Instruction Set

Instruction ID	Instruction	Instruction Code	Instruction Type
O	bneq	0000	I
J	srl	0001	S
N	beq	0010	I
C	sub	0011	R
A	add	0100	R
E	and	0101	R
P	j	0110	J
L	lw	0111	I
G	or	1000	R
K	nor	1001	R
M	sw	1010	I
F	andi	1011	I
D	subi	1100	I
B	addi	1101	I
H	ori	1110	I
I	sll	1111	S

ICs and Components Used

- Registers — 8 (one 8-bit, seven 4-bit)
- Adders — 3 (two 8-bit, one 4-bit)
- Multiplexers — 8 (one 8-bit 2x1, two 4-bit 16x1, one 4-bit 8x1, three 4-bit 2x1)
- Demultiplexer — 1
- ROM — 2
- RAM — 1
- Shifters — 2
- Bit extenders — 6
- Splitters — 7
- XOR gate — 2
- AND gate — 2
- OR gate — 1
- NOR gate — 2
- Constant signal — 9
- Clock — 1

Logisim Version: Logisim Evolution v4.1.0

How to write and execute a program on our circuit:

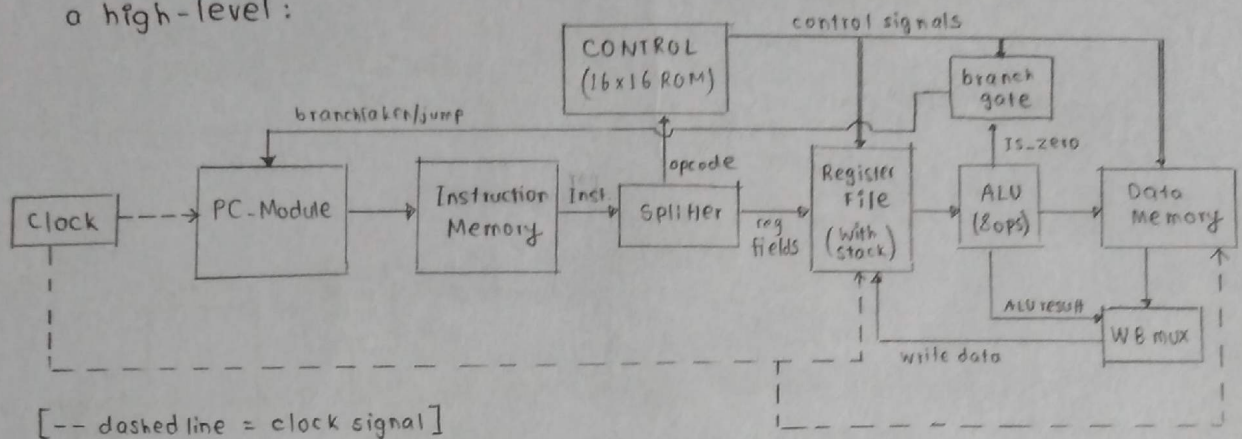
1. Write raw instructions in MIPS code in a test.txt file (any name works)
2. Use the assembler.py and run this command in terminal in one line:

`python assembler.py test.txt`

 (ensure both files in same directory)
3. An output file called "instruction-memory.txt" will be generated with the 16-bit converted codes.
4. In Logisim Evolution, click on the Instruction-Memory module, "right-click on ROM > Load Image > select "instruction-memory.txt"
5. Reset Simulation first using "Simulate > Reset Simulation".
6. Poke the CLK component to advance manually, or enable Auto-Tick with Ctrl/Cmd + K.
7. Check the outputs in the main circuit (these are labelled by component).

Architecture and Approach Overview:

Our approach consists of the following component groups at a high-level:



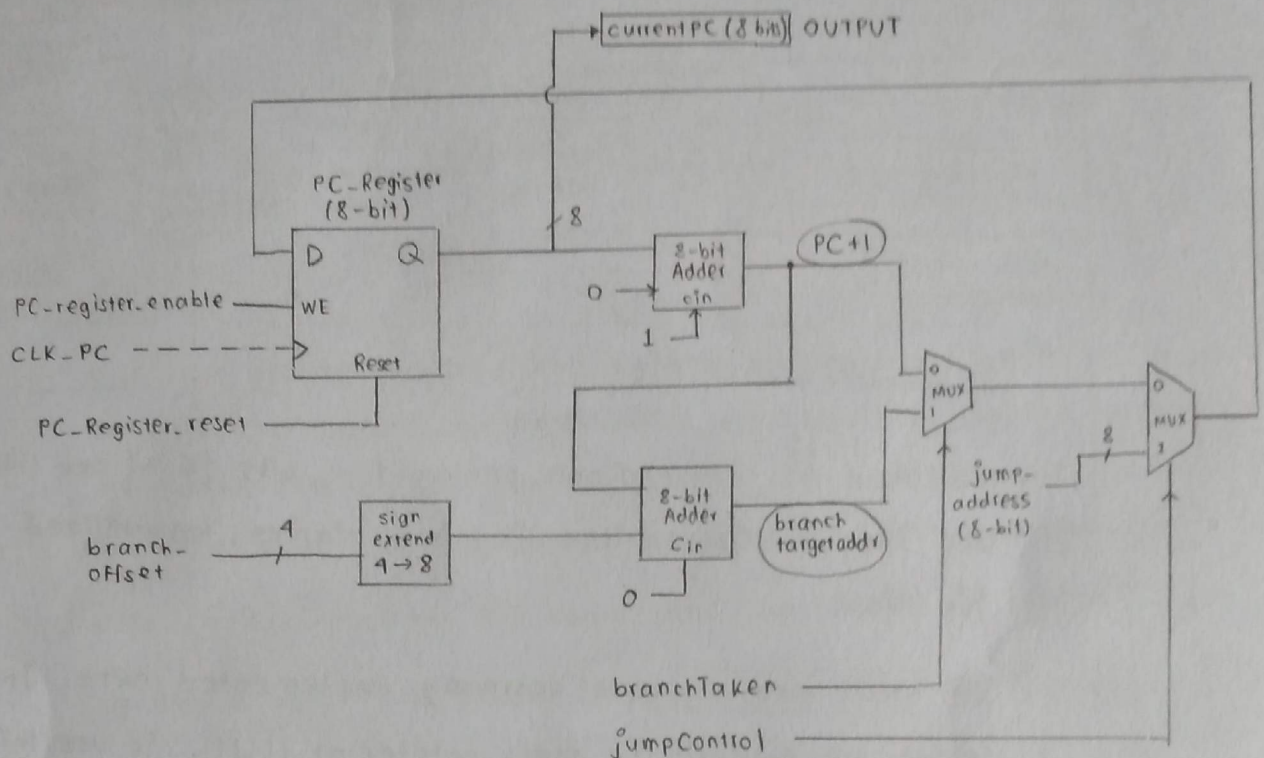
- We used ¹²~~13~~ control signals per word — bits [6-3] are left as spare, and the ALUOp requires 3 bits of signals, we utilized bits [2-0] for this.
- The **stack memory** was correctly implemented here. In our design, we initiate the stack pointer at **1111**. To use this reliably as the first command/instruction in every simulation, we modified our assembler script to append an additional instruction at address 00 of every program:

`addi $sp, $zero, 1111 → 1101 0000 0110 1111 → D06F`

- A single input pin **RST** in the main circuit can be manually used to :
 - clear the program counter to 00, and
 - every register to 0000
- A single clock **CLK** distributes signals to three modules that contain storage elements : **PC-Module**, **Register-File**, **MEM**; phases are guaranteed not to mismatch.
- The data-memory RAM is configured as **Falling-edge-triggered**, the others are **rising-edge-triggered**.

The Individual Modules

① PC Module



- The register's output is the current PC and is driven out into instruction memory.
- The first MUX chooses between next sequential instruction PC+1 and the branch address.
- The second MUX chooses between this and 8-bit jump address, determined by 'jumpControl' signal.

Example dry run:

bneq \$t1, \$zero, loop at address 07, let \$t1 is not zero.

Before rising edge,

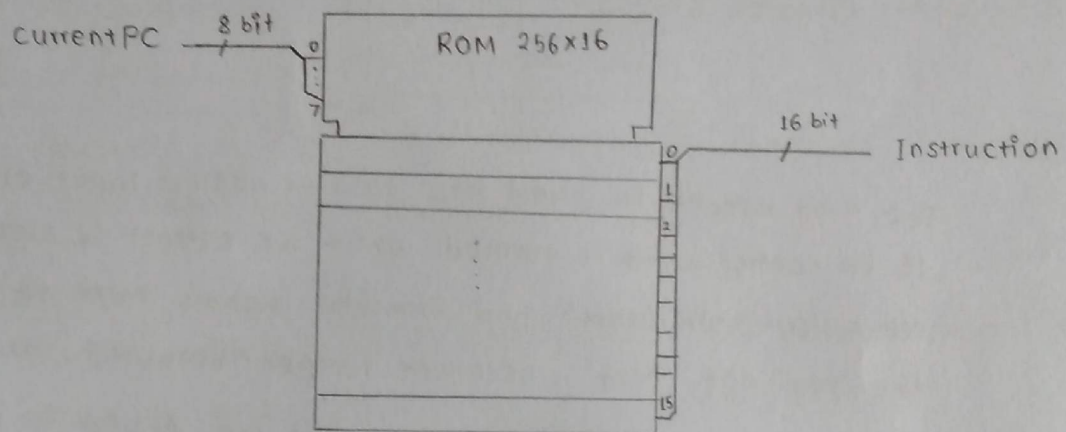
Signal / Element	Value
Current PC	0000 0111
Adder_1 output	0000 1000 = PC+1
branch_offset	1011 → 1111 1011
Adder_2 output	0000 0011 → loop addr
branchTaken	1
MUX 1 out	0000 0011
jumpControl	0
Mux 2 out	0000 0011
PC-register.enable	1

After clock pulse : current PC becomes 3 (0000 0011).

adder 1 $\rightarrow$ 04

adder 2 $\rightarrow$ new target

② Instruction Memory

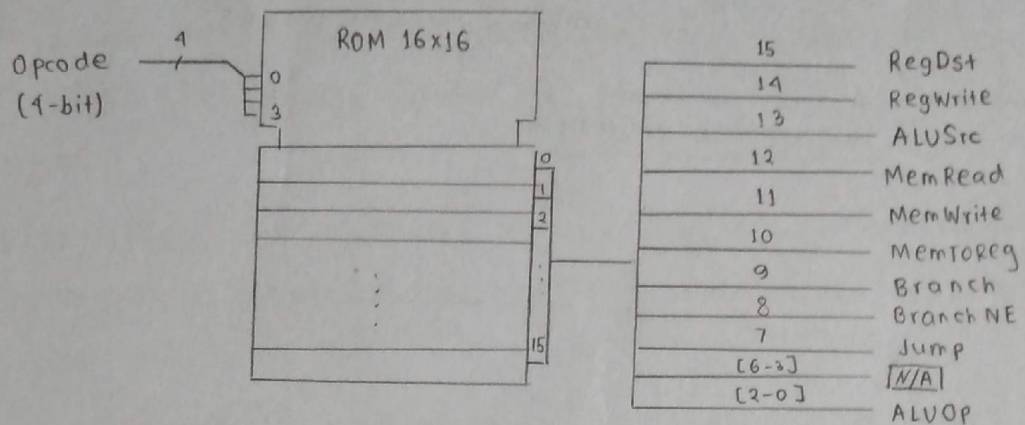


- When the rising edge changes the program counter, the new instruction immediately appears at the output of this ROM.

Example:

Input (current PC)	Output instruction	MIPS equivalent
0000 0011 (03)	0100 0011 0010 0011 (4323)	add \$t2, \$t2, \$t1
0000 0101 (05)	1010 0110 0010 0000 (A620)	sw \$t1, 0(\$sp)
0000 0111 (07)	0000 0010 0000 1011 (020B)	bneq \$t1, \$zero, loop

③ Control Unit

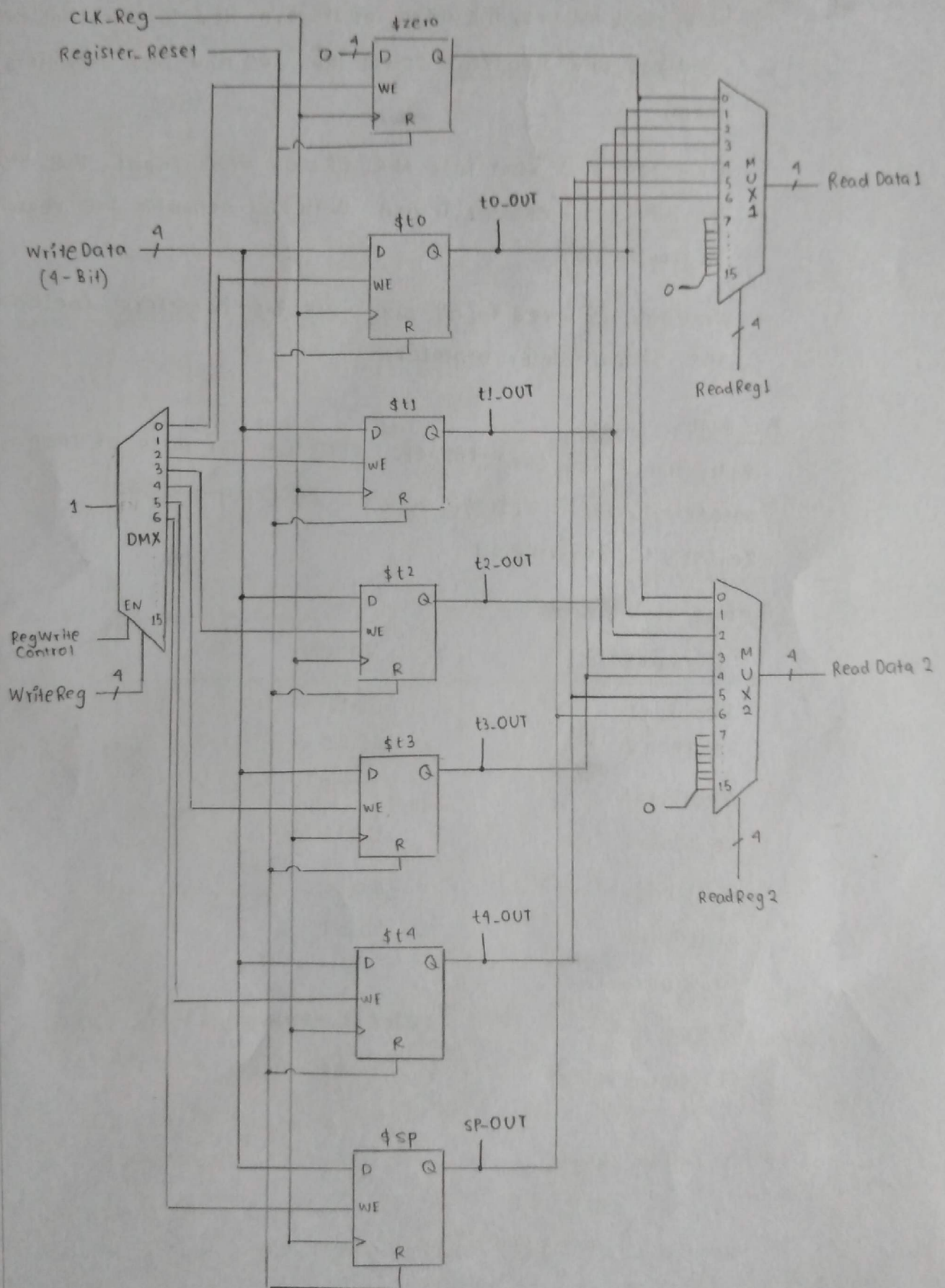


The 4-bit opcode is wired into data or address input of ROM. A 16-bit control word is emitted, which we extract 12 signals from. We output both Branch and BranchNE signals here to act on the 'beq' and 'bneq' operations respectively. We take dont-care = 0. Bits [6-3] have been left spare, and the ALVOP is bits [2-0].

The signal activations in our design follow the table below:

Addr	Instr	Word	RegDst	RegWrite	ALUSrc	MemRead	MemWrite	MemToReg	Branch	BranchNE	Jump	ALVOp
0	bneq	0301	0	0	0	0	0	0	1	1	0	001 sub
1	srl	6006	0	1	1	0	0	0	0	0	0	110 srl
2	beq	0201	0	0	0	0	0	0	1	0	0	001 sub
3	sub	c001	1	1	0	0	0	0	0	0	0	001 sub
4	add	c002	1	1	0	0	0	0	0	0	0	000 add
5	and	0087	1	1	0	0	0	0	0	0	0	010 and
6	j	7400	0	0	0	0	0	0	0	0	1	111 const
7	lw	c003	0	1	1	1	0	1	0	0	0	000 add
8	or	c004	1	1	0	0	0	0	0	0	0	011 or
9	nor	2800	1	1	0	0	0	0	0	0	0	100 nor
10	sw	6002	0	0	1	1	1	0	0	0	0	000 add
11	andi	6001	0	1	1	0	0	0	0	0	0	010 and
12	subi	6000	0	1	1	0	0	0	0	0	0	001 sub
13	addi	6003	0	1	1	0	0	0	0	0	0	000 add
14	ori	6005	0	1	1	0	0	0	0	0	0	011 or
15	sll	c000	0	1	1	0	0	0	0	0	0	101 sll

④ Register File



Here,

- the seven register outputs are wired to inputs 0 through 6 on both read MUXes, the other inputs are tied to constant zero.
ReadReg1 and ReadReg2 select the SRC and DST registers from them.
- A constant 1 is sent into the DEMUX data input. The signal RegWriteControl enables it and WriteReg activates the required register's line.
- WriteData is wired to all six writeable registers, including the Stack Pointer register.

Example:

Instruction 4323 (add \$t2, \$t2, \$t1) on 1st pass of loop.

opcode = 4, src1 = 3 (\$t2), src2 = 2 (\$t1), dst = 3 (\$t2).

RegDst = 1, RegWrite = 1.

Before rising edge:

Element	Value
ReadReg1	0011 → \$t2
ReadReg2	0010 → \$t1
ReadData1	0000
ReadData2	0011
WriteReg	0011
WriteData	0011
RegWriteControl	1
DEMUX out	line 3 enabled (\$t2)
\$t2 stored value	0000

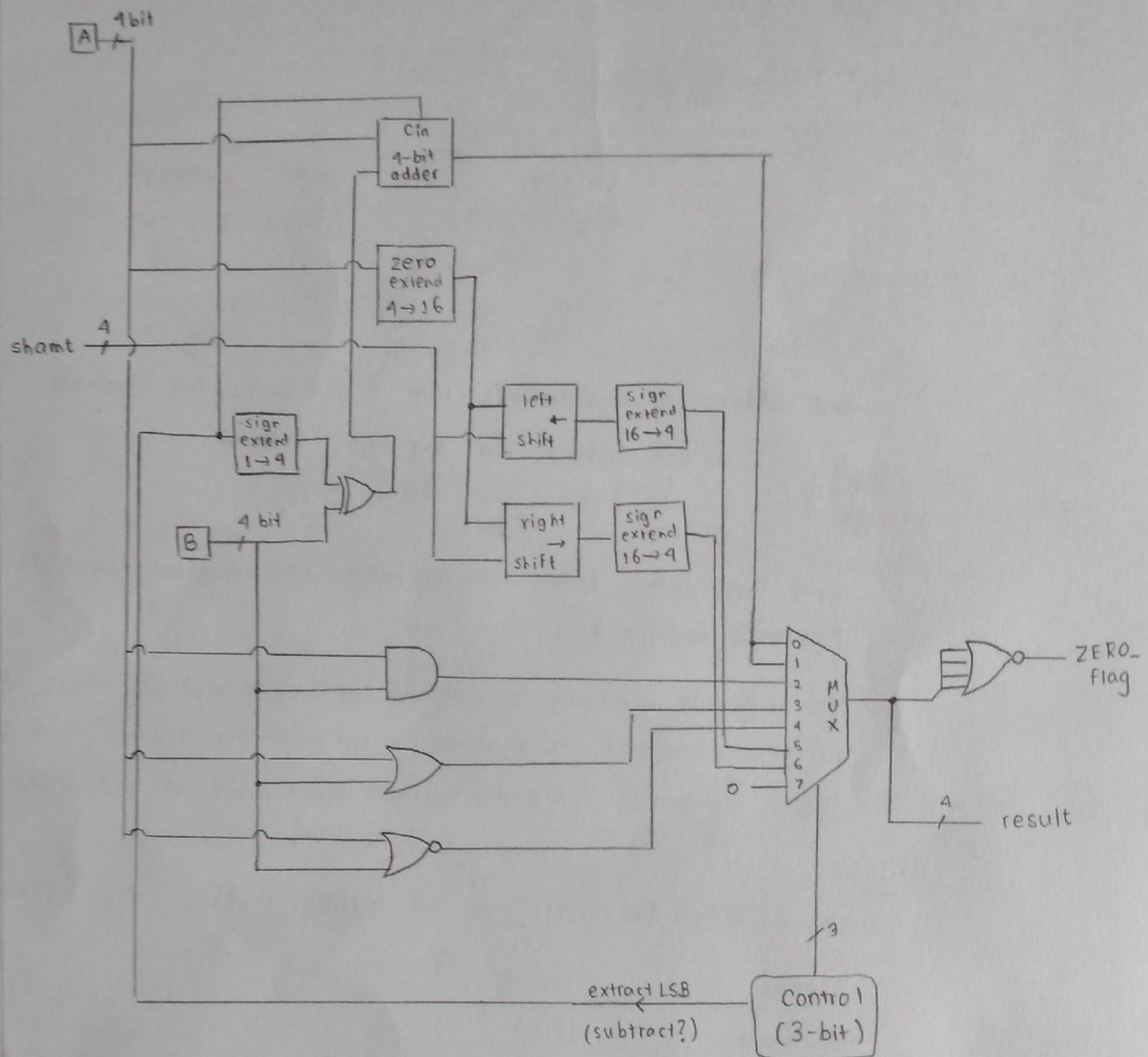
After rising edge:

\$t2 → 0011

ReadData1 → 0011

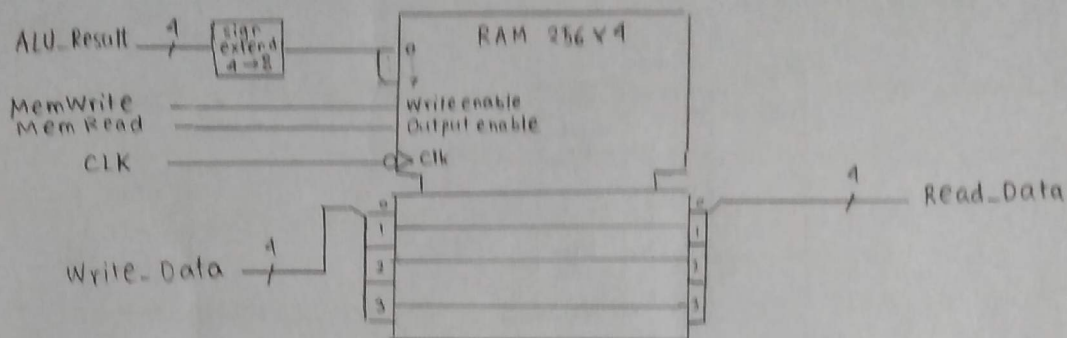
New instruction appears at DEMUX.

⑤ ALU



- The 3-bit control signal from ALUOp drives the 8-to-1 MUX selection of operation (0: add, 1: sub, 2: AND, 3: OR, 4: NOR, 5: sll, 6: srl, 7: const).
- The LSB of ALUOp is extracted to invert B and $C_{in} = 1$ to mimic subtraction.
- The shifters are widened to 16 bits first, then truncated back to 4 bits for correct behaviour.
- The 4-input NOR gate directly gives us an all-zero check, which is used for branch gating in the main circuitry.

⑥ Data and Stack Memory



- The address always arrives from ALU-Result. For example:

$lw \$t3, 0(\$sp) \rightarrow \$sp + 0$

$sw \$t1, 2(\$zero) \rightarrow 0 + 2$

- Write-Data comes from register file's 2nd read port and does not pass through ALU.
- ! No separate memory is needed for implementing stack, even no new opcodes. Stack access = an ordinary lw/sw that uses $\$sp$ as the base. The stack grows downward as $\$sp$ decrements.

Example:

Instr A620 ($sw \$t1, 0(\$sp)$) on 3rd iteration, $\$sp = 1100$, $\$t1 = 0001$

Signal	Value
ALU-Result	1100
WRITE_DATA	0001
MEM_WRITE	1
MEM_READ	0

After clock pulse, on falling edge, $RAM[12]$ becomes 0001.

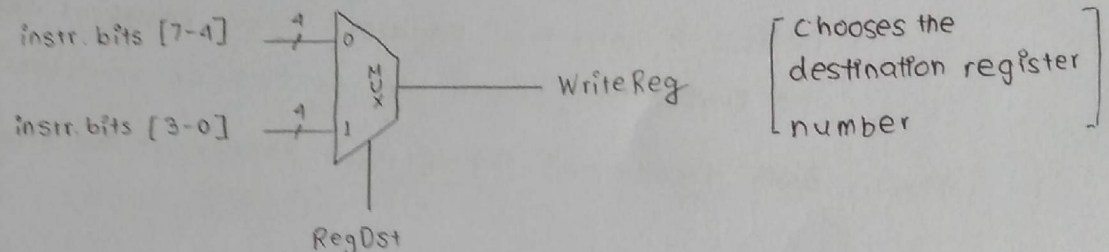
How the whole circuit works

In the main circuit, there are some additional elements that connect the individual modules and provide required logic. These are:

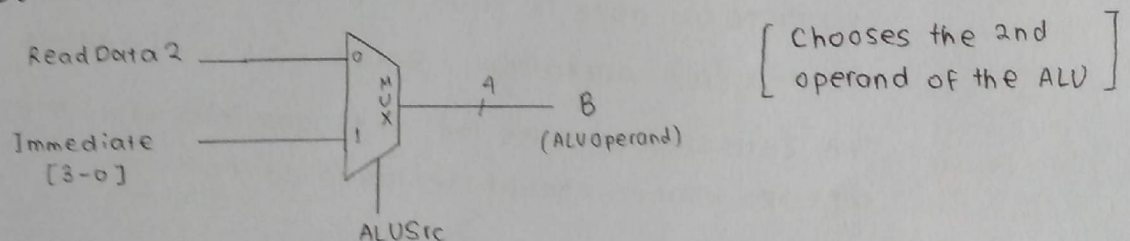
- Instruction field splitter — this breaks the 16-bit instruction into four 4-bit fields: 15-12 (opcode), 11-8 (field 2), 7-4 (field 3) and 3-0 (field 4).

- Jump address splitter — combines the bits 11-4 as an 8-bit value, which is the J-type target address.

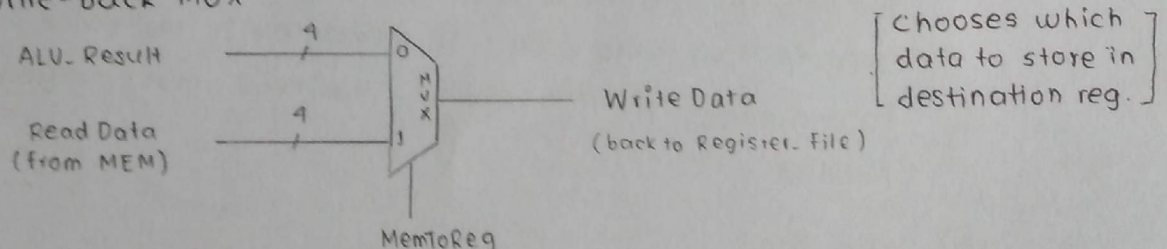
- RegDst MUX —



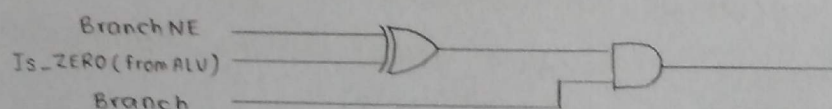
- ALUSrc MUX —



- Write-back MUX —



- Branch gate



The computed branchTaken signal is calculated as:

$$\text{branchTaken} = \text{Branch AND (Is_ZERO XOR BranchNE)}$$

Five-stage working mechanism

① Instruction Fetch

The clock's rising edge loads the program counter. Its output addresses the instruction memory ROM, which produces the 16-bit instruction combinationally. The two splitters redirect the bits into small groups as required.

The PC module begins computing $PC+1$, and branch target for the instruction.

② Instruction Decode

The opcode $\rightarrow$ enters control ROM $\rightarrow$ emits 16-bit control word; splitter $\rightarrow$ gives 12 separate signals. At the same time, fields 2, 3 enter the Register File as ReadReg1 and ReadReg2. The RegDst MUX selects the destination register number.

Both decode and registerread happen at the same time.

③ Execute

The ALUSrc signal chooses operand B (ReadData2/Immdt.). All eight operations are done in parallel in ALU and ALUOp (3 bits) selects the required operation. Result = answer / address.

The Zero Flag emerges too, and goes into branch gate to decide whether branch should be taken or not.

④ Memory

The ALU result goes to data memory as an address, and ReadData2 goes to it as write data. If MemWrite = 1, the store commits at falling edge; if MemRead = 1, the read data emerges with no edge needed.

⑤ Write Back

The write-back MUX selects either the memory data or the ALU Result and outputs as WriteData back to Register File.

If Regwrite = 1, the write-deMUX enables one register to wait to be written in by the next cycle.

Example Program

Let's trace the following MIPS code below to understand the values flowing through the circuitry.

Addr	Assembly code	Machine Code
00	addi \$sp, \$zero, 1111 (15)	D06F
01	addi \$t1, \$zero, 3	D023
02	addi \$t2, \$zero, 0	D030
03	loop: add \$t2, \$t2, \$t1	4323
04	subi \$sp, \$sp, 1	C661
05	sw \$t1, 0(\$sp)	A620
06	subi \$t1, \$t1, 1	C221
07	bneq \$t1, \$zero, loop	020B
08	lw \$t3, 0(\$sp)	7640
09	add \$t0, \$t2, \$t3	4341
10	beq \$t0, \$zero, skip	2101
11	sll \$t0, \$t0, 1	F111
12	skip: j done	60E0
13	ori \$t0, \$t0, 0	E110
14	done: srl \$t0, \$t0, 1	1111
15	halt: j halt	60F0

Expected values: \$t0 = 0111, \$t2 = 0110, \$t3 = 0001, \$sp = 1100

Execution Trace

PC	Instr	Op	Reg Dst	Reg Write	ALUSrc	Mem Read	Mem Write	Mem to Reg	Branch	BNE	J	ALU A	ALU B	ALU Result	Is zero	branch Taken	Commits at clock edge	Next PC
0	D06F	addi	0	1	1	0	0	0	0	0	0	0000	1111	1111	0	0	\$sp ← 1111	1
1	D023	addi	0	1	1	0	0	0	0	0	0	0000	0011	0011	0	0	\$t1 ← 0011	2
2	D030	addi	0	1	1	0	0	0	0	0	0	0000	0000	0000	1	0	\$t2 ← 0000	3
3	4323	add	1	1	0	0	0	0	0	0	0	0000	0011	0011	0	0	\$t2 ← 0011	4
4	C661	subi	0	1	1	0	0	0	0	0	0	1111	0001	1110	0	0	\$sp ← 1110	5
5	A620	sw	0	0	1	0	1	0	0	0	0	1110	0000	1110	0	0	RAM[14] ← 0011	6
6	C221	subi	0	1	1	0	0	0	0	0	0	0011	0001	0010	0	0	\$t1 ← 0010	7
7	020B	bneq	0	0	0	0	0	0	1	1	0	0010	0000	0010	0	1	-	3
3	4323	add	1	1	0	0	0	0	0	0	0	0011	0010	0101	0	0	\$t2 ← 0101	4
4	C661	subi	0	1	1	0	0	0	0	0	0	1110	0001	1110	0	0	\$sp ← 1110	5
5	A620	sw	0	0	1	0	1	0	0	0	0	1101	0000	1101	0	0	RAM[13] ← 0010	6
6	C221	subi	0	1	1	0	0	0	0	0	0	0010	0001	0001	0	0	\$t1 ← 0001	7
7	020B	bneq	0	0	0	0	0	0	1	1	0	0001	0000	0001	0	1	-	3
3	4323	add	1	1	0	0	0	0	0	0	0	0101	0001	0110	0	0	\$t2 ← 0110	4
4	C661	subi	0	1	1	0	0	0	0	0	0	1101	0001	1100	0	0	\$sp ← 1100	5
5	A620	sw	0	0	1	0	1	0	0	0	0	1100	0000	1100	0	0	RAM[12] ← 0001	6
6	C221	subi	0	1	1	0	0	0	0	0	0	0001	0001	0000	1	0	\$t1 ← 0000	7
7	020B	bneq	0	0	0	0	0	0	1	1	0	0000	0000	0000	1	0	→ loop exit	8
8	7640	lw	0	1	1	1	0	1	0	0	0	1100	0000	1100	0	0	\$t3 ← 0001	9
9	4341	add	1	1	0	0	0	0	0	0	0	0110	0001	0111	0	0	\$t0 ← 00111	A
A	2101	beq	0	0	0	0	0	0	1	0	0	0111	0000	0111	0	0	→ not taken	B
B	F111	sll	0	1	1	0	0	0	0	0	0	0111	0001	1110	0	0	\$t0 ← 1110	C
C	60E0	j	0	0	0	0	0	0	0	0	1	-	-	0000	1	0	skips D	E
E	1111	srl	0	1	1	0	0	0	0	0	0	1110	0001	0111	0	0	\$t0 ← 0111	F
F	60F0	j	0	0	0	0	0	0	0	0	1	-	-	0000	1	0	halt	F

Discussion

Q. Why is the control word 16 bits wide when only 12 signals are used?

Ans: A 16-bit word gives a clean four hexadecimal digits, so every control word can be read and split easily. The overhead for including spare bits is negligible compared to the flexibility it provides if we want to add extra functionality in the future, and hence more required control signals.

Q. Why are Branch and BranchNE two separate signals instead of a single PCSic field?

Ans: Because the Control Unit cannot decide by itself whether to branch or not — it also depends on the ALU's zero-flag, which emerges only after the operands have been compared.

The actual decision is made through a branch gating mechanism involving a simple XOR and an AND gate. Thus, both instructions can share the same datapath, and only one gets reliably picked.

Q. Why does the data memory store on the falling edge, while every register reads/captures on rising edge?

Ans: This separates the two events in time within a single cycle. If the memory wrote during the register/PC read moment, the store would race the change of instruction. By writing at the falling edge, the store is committed when the address or data have been stable for a long time.

Q. How is the stack memory implemented without separate push/pop instructions?

Ans: Since all 16 opcodes were allocated by the instruction set, we made it such that a stack access is simply an ordinary lw/sw that happens to use \$sp as the base register. The stack occupies the top of the same data memory, initialized at 1111, and grows downward. Our assembler expands push/pop into two-instruction sequences, so the hardware does not need to care about the code.